

CAS PRATIQUE DE LA POO

Programmation Orientee Objet en PHP

Applicatiquer à un Systeme de Facturation

Langage : PHP 8.x
Domaine : Programmation Orientee Objet
Application : Systeme de Facturation

0 Table des matières

1	Introduction	2
1.1	Organisation du projet	2
1.2	Contexte de l'application	2
2	Etape 1 : Declaration d'une classe	3
2.1	Concept : Qu'est-ce qu'une classe ?	3
2.2	Application : La classe Produit	3
3	Etape 2 : Attributs et methodes	4
3.1	Concept : Attributs et methodes	4
3.2	Application : Ajout des attributs et methodes a Produit	4
3.3	Utilisation : Creation d'un objet	5
4	Etape 3 : Le constructeur	6
4.1	Concept : Le constructeur	6
4.2	Application : Constructeur dans Produit et creation de Client	6
4.3	Utilisation avec le constructeur	7
5	Etape 4 : Interconnexion des fichiers	9
5.1	Concept : require et require_once	9
5.2	Application : Creation de LigneFacture	9
5.3	Application : Creation de la classe Facture	10
5.4	Utilisation complete — index.php	12
6	Etape 5 : Encapsulation	14
6.1	Concept : L'encapsulation	14
6.2	Application : Produit avec encapsulation	14
6.3	Demonstration de la protection offerte par l'encapsulation	16
7	Etape 6 : Heritage	17
7.1	Concept : L'heritage	17
7.2	Application : FactureDetaillee herite de Facture	17
7.3	Utilisation	19
8	Etape 7 : Polymorphisme	21
8.1	Concept : Le polymorphisme	21
8.2	Application : Demonstration du polymorphisme	21
9	Etape 8 : Abstraction	23
9.1	Concept : Les classes abstraites et les interfaces	23
9.2	Application : Entite abstraite commune	23
9.3	Application : Produit et Client heritent de EntiteAbstraite	24
9.4	Demonstration de l'abstraction	27
10	Recapitulatif des concepts	29
11	Structure finale du projet	29

1 Introduction

Ce cas pratique a pour objectif d'introduire les concepts fondamentaux de la programmation orientee objet (POO) en PHP au travers d'une application concrete : un systeme de facturation.

Nous construirons ce systeme etape par etape, en introduisant un concept de la POO a chaque etape. A la fin de ce document, vous aurez une application fonctionnelle capable de gerer des clients, des produits, et de generer des factures.

1.1 Organisation du projet

Le projet sera organise dans la structure de fichiers suivante :

```
1 facturation/  
2 |-- Produit.php  
3 |-- Client.php  
4 |-- LigneFacture.php  
5 |-- Facture.php  
6 |-- FactureDetaillee.php  
7 |-- EntiteAbstraite.php  
8 |-- index.php
```

Listing 1 – Structure du projet

1.2 Contexte de l'application

Notre systeme de facturation permettra de :

- Creer des produits avec un nom, un prix unitaire et une quantite en stock
- Creer des clients avec leurs coordonnees
- Generer des factures contenant des lignes de produits
- Calculer le total d'une facture avec ou sans TVA
- Afficher les details d'une facture

2 Etape 1 : Declaration d'une classe

2.1 Concept : Qu'est-ce qu'une classe ?

Definition. Une classe est un modele ou un plan de construction qui decrit la structure et le comportement d'un objet. Elle regroupe des donnees (appelees attributs ou proprietes) et des comportements (appeles methodes ou fonctions). En PHP, une classe se declare avec le mot-cle `class`.

On peut comparer une classe a un moule : le moule definit la forme, et chaque objet fabrique avec ce moule est une instance de cette classe.

Syntaxe generale :

```
1 <?php
2 class NomDeLaClasse
3 {
4     // Les attributs et methodes vont ici
5 }
```

Listing 2 – Syntaxe de declaration d'une classe

2.2 Application : La classe Produit

Nous allons creer notre premiere classe : `Produit`. Un produit possede un nom, un prix unitaire et une quantite en stock.

```
1 <?php
2 // Fichier : Produit.php
3
4 class Produit
5 {
6     // Pour l'instant, la classe est vide.
7     // Nous allons l'enrichir dans les etapes suivantes.
8 }
```

Listing 3 – Fichier `Produit.php` — Declaration basique d'une classe

Remarque. En PHP, le nom d'une classe commence conventionnellement par une majuscule. Chaque classe est generalement placee dans son propre fichier, dont le nom correspond au nom de la classe (par exemple `Produit.php` pour la classe `Produit`).

3 Etape 2 : Attributs et methodes

3.1 Concept : Attributs et methodes

Les attributs sont des variables definies a l'interieur d'une classe. Ils representent l'etat ou les caracteristiques d'un objet. Par exemple, un produit a un nom, un prix et une quantite.

Les methodes sont des fonctions definies a l'interieur d'une classe. Elles representent les comportements ou les actions que peut effectuer un objet. Par exemple, un produit peut afficher ses informations ou calculer sa valeur en stock.

En PHP, les attributs et methodes sont declares avec des modificateurs de visibilite (`public`, `private`, `protected`). Nous utiliserons d'abord `public` pour que tout soit accessible, puis nous verrons les nuances avec l'encapsulation.

3.2 Application : Ajout des attributs et methodes a Produit

```
1 <?php
2 // Fichier : Produit.php
3
4 class Produit
5 {
6     // ---- Attributs ----
7     // Ce sont les caracteristiques d'un produit
8     public string $nom;
9     public float $prixUnitaire;
10    public int $quantiteStock;
11
12    // ---- Methodes ----
13
14    /**
15     * Affiche les informations du produit
16     */
17    public function afficherInfos(): void
18    {
19        echo "Produit : " . $this->nom . "\n";
20        echo "Prix unitaire : " . $this->prixUnitaire . " CDF\n";
21        echo "Stock : " . $this->quantiteStock . " unites\n";
22    }
23
24    /**
25     * Calcule la valeur totale du stock pour ce produit
26     */
27    public function valeurStock(): float
28    {
29        return $this->prixUnitaire * $this->quantiteStock;
30    }
31 }
```

Listing 4 – Fichier Produit.php — Avec attributs et methodes

Le mot-cle \$this. A l'interieur d'une methode, `$this` est une reference vers l'objet courant. Il permet d'accéder aux attributs et aux autres methodes de l'objet. Par exemple, `$this->nom` fait reference a l'attribut `nom` de l'objet en cours d'utilisation.

3.3 Utilisation : Creation d'un objet

```
1 <?php
2 // Fichier : index.php
3
4 // Inclusion de la classe Produit
5 require_once 'Produit.php';
6
7 // Creation d'un objet (instance) de la classe Produit
8 $produit1 = new Produit();
9
10 // Affectation des valeurs aux attributs
11 $produit1->nom = "CHAWARMA POULET";
12 $produit1->prixUnitaire = 6650.99;
13 $produit1->quantiteStock = 15;
14
15 // Utilisation des methodes
16 $produit1->afficherInfos();
17
18 echo "Valeur totale en stock : " . $produit1->valeurStock() . " CDF\n";
```

Listing 5 – Fichier index.php — Creation et utilisation d'un objet Produit

Resultat attendu :

```
1 Produit : Clavier mecanique
2 Prix unitaire : 6650.99 CDF
3 Stock : 15 unites
4 Valeur totale en stock : 99764.85 CDF
```

Listing 6 – Sortie de l'execution

4 Etape 3 : Le constructeur

4.1 Concept : Le constructeur

Definition. Le constructeur est une methode speciale qui est appelee automatiquement lors de la creation d'un objet avec le mot-cle `new`. En PHP, le constructeur s'appelle toujours `__construct()`. Il permet d'initialiser les attributs d'un objet des sa creation, evitant ainsi d'avoir a les renseigner un par un apres l'instanciation.

Le constructeur peut recevoir des parametres, exactement comme une fonction ordinaire.

4.2 Application : Constructeur dans Produit et creation de Client

```
1 <?php
2 // Fichier : Produit.php
3
4 class Produit
5 {
6     public string $nom;
7     public float $prixUnitaire;
8     public int $quantiteStock;
9
10    /**
11     * Constructeur : initialise le produit des sa creation
12     *
13     * @param string $nom          Nom du produit
14     * @param float $prixUnitaire  Prix HT unitaire
15     * @param int $quantiteStock  Quantite disponible en stock
16     */
17    public function __construct(
18        string $nom,
19        float $prixUnitaire,
20        int $quantiteStock = 0
21    ) {
22        $this->nom          = $nom;
23        $this->prixUnitaire = $prixUnitaire;
24        $this->quantiteStock = $quantiteStock;
25    }
26
27    public function afficherInfos(): void
28    {
29        echo "Produit : " . $this->nom . "\n";
30        echo "Prix unitaire : " . $this->prixUnitaire . " CDF\n";
31        echo "Stock : " . $this->quantiteStock . " unites\n";
32    }
33
34    public function valeurStock(): float
35    {
36        return $this->prixUnitaire * $this->quantiteStock;
37    }
```

38 }

Listing 7 – Fichier Produit.php — Avec constructeur

Nous creons maintenant une classe `Client` avec son propre constructeur :

```
1 <?php
2 // Fichier : Client.php
3
4 class Client
5 {
6     public string $nom;
7     public string $prenom;
8     public string $email;
9     public string $adresse;
10
11     /**
12      * Constructeur de Client
13      */
14     public function __construct(
15         string $nom,
16         string $prenom,
17         string $email,
18         string $adresse
19     ) {
20         $this->nom      = $nom;
21         $this->prenom   = $prenom;
22         $this->email    = $email;
23         $this->adresse  = $adresse;
24     }
25
26     /**
27      * Retourne le nom complet du client
28      */
29     public function getNomComplet(): string
30     {
31         return $this->prenom . " " . $this->nom;
32     }
33
34     public function afficherInfos(): void
35     {
36         echo "Client : " . $this->getNomComplet() . "\n";
37         echo "Email : " . $this->email . "\n";
38         echo "Adresse : " . $this->adresse . "\n";
39     }
40 }
```

Listing 8 – Fichier Client.php — Classe Client avec constructeur

4.3 Utilisation avec le constructeur

1 <?php


```
2 require_once 'Produit.php';
3 require_once 'Client.php';
4
5 // Creation directe avec les valeurs passees au constructeur
6 $produit1 = new Produit("Clavier mecanique", 89.99, 15);
7 $produit2 = new Produit("Souris sans fil", 45.50, 30);
8
9 $client1 = new Client("Ezechiel", "Itewe", "E.itewe@email.com",
10                      "12 Bandalugwa ,RDC Bonka Elengi");
11
12 $produit1->afficherInfos();
13 echo "\n";
14 $client1->afficherInfos();
```

Listing 9 – Fichier index.php — Creation avec constructeur

5 Etape 4 : Interconnexion des fichiers

5.1 Concept : require et require_once

Definition. En PHP, `require` et `require_once` permettent d'inclure le contenu d'un fichier dans un autre. Cette pratique est essentielle en POO car chaque classe est placee dans son propre fichier.

La difference entre `require` et `require_once` :

- `require` inclut le fichier a chaque fois qu'il est appele. Si la classe est incluse deux fois, PHP genere une erreur (redeclaration de classe).
- `require_once` verifie si le fichier a deja ete inclus. Si c'est le cas, il ne l'inclut pas une seconde fois. C'est la methode recommandee pour les classes.

5.2 Application : Creation de LigneFacture

Une ligne de facture represente un produit commande avec sa quantite. Elle fait reference a un objet `Produit` — c'est ce qu'on appelle la composition.

```
1 <?php
2 // Fichier : LigneFacture.php
3
4 // Ce fichier depend de Produit.php
5 require_once 'Produit.php';
6
7 class LigneFacture
8 {
9     public Produit $produit;
10    public int $quantite;
11
12    /**
13     * Constructeur : une ligne de facture associe un produit
14     * a une quantite commandee.
15     *
16     * @param Produit $produit L'objet Produit concerne
17     * @param int     $quantite La quantite commandee
18     */
19    public function __construct(Produit $produit, int $quantite)
20    {
21        $this->produit = $produit;
22        $this->quantite = $quantite;
23    }
24
25    /**
26     * Calcule le sous-total de cette ligne
27     * (prix unitaire x quantite commandee)
28     */
29    public function calculerSousTotal(): float
30    {
31        return $this->produit->prixUnitaire * $this->quantite;
32    }
33
```

```

34  /**
35   * Affiche le detail de la ligne
36   */
37  public function afficher(): void
38  {
39      $sousTotal = $this->calculerSousTotal();
40      echo sprintf(
41          "%-30s | %5d | %8.2f CDF | %10.2f CDF\n",
42          $this->produit->nom,
43          $this->quantite,
44          $this->produit->prixUnitaire,
45          $sousTotal
46      );
47  }
48  }

```

Listing 10 – Fichier LigneFacture.php — Interconnexion avec Produit

5.3 Application : Creation de la classe Facture

```

1  <?php
2  // Fichier : Facture.php
3
4  // Inclusion des dependances
5  require_once 'Client.php';
6  require_once 'LigneFacture.php';
7
8  class Facture
9  {
10     public string $numero;
11     public string $dateCreation;
12     public Client $client;
13     public array $lignes; // Tableau de LigneFacture
14     public float $tauxTva; // Exemple : 0.16 pour 16%
15
16     /**
17      * Constructeur de Facture
18      */
19     public function __construct(
20         string $numero,
21         Client $client,
22         float $tauxTva = 0.16
23     ) {
24         $this->numero = $numero;
25         $this->client = $client;
26         $this->tauxTva = $tauxTva;
27         $this->lignes = [];
28         $this->dateCreation = date('d/m/Y');
29     }
30
31     /**
32      * Ajoute une ligne de facture

```

```

33      */
34      public function ajouterLigne(LigneFacture $ligne): void
35      {
36          $this->lignes[] = $ligne;
37      }
38
39      /**
40       * Calcule le total HT (hors taxes)
41       */
42      public function calculerTotalHT(): float
43      {
44          $total = 0.0;
45          foreach ($this->lignes as $ligne) {
46              $total += $ligne->calculerSousTotal();
47          }
48          return $total;
49      }
50
51      /**
52       * Calcule le montant de la TVA
53       */
54      public function calculerTVA(): float
55      {
56          return $this->calculerTotalHT() * $this->tauxTva;
57      }
58
59      /**
60       * Calcule le total TTC (toutes taxes comprises)
61       */
62      public function calculerTotalTTC(): float
63      {
64          return $this->calculerTotalHT() + $this->calculerTVA();
65      }
66
67      /**
68       * Affiche la facture complete
69       */
70      public function afficher(): void
71      {
72          echo str_repeat("=", 65) . "\n";
73          echo "FACTURE N : " . $this->numero . "\n";
74          echo "Date      : " . $this->dateCreation . "\n";
75          echo "Client     : " . $this->client->getNomComplet() . "\n";
76          echo "Adresse    : " . $this->client->adresse . "\n";
77          echo str_repeat("-", 65) . "\n";
78          echo sprintf(
79              "%-30s | %5s | %12s | %12s\n",
80              "Produit", "Qte", "Prix U.", "Sous-total"
81          );
82          echo str_repeat("-", 65) . "\n";
83

```

```

84         foreach ($this->lignes as $ligne) {
85             $ligne->afficher();
86         }
87
88         echo str_repeat("-", 65) . "\n";
89         echo sprintf("%-48s %10.2f CDF\n", "Total HT :", $this->
calculerTotalHT());
90         echo sprintf("%-48s %10.2f CDF\n",
91             "TVA (" . ($this->tauxTva * 100) . "%) :", $this->calculerTVA())
;
92         echo str_repeat("-", 65) . "\n";
93         echo sprintf("%-48s %10.2f CDF\n", "TOTAL TTC :", $this->
calculerTotalTTC());
94         echo str_repeat("=", 65) . "\n";
95     }
96 }

```

Listing 11 – Fichier Facture.php — Classe principale Facture

5.4 Utilisation complete — index.php

```

1 <?php
2 // Fichier : index.php
3
4 // Inclusion des classes necessaires
5 // (Facture.php inclut deja Client.php et LigneFacture.php,
6 // qui inclut lui-meme Produit.php)
7 require_once 'Facture.php';
8
9 // Creation des produits
10 $produit1 = new Produit("Clavier mecanique", 89.99, 15);
11 $produit2 = new Produit("Souris sans fil", 45.50, 30);
12 $produit3 = new Produit("Ecran 27 pouces", 349.00, 5);
13
14 // Creation du client
15 $client = new Client("Ezechiel", "ITEWE", "ezechiel.itewe@email.com",
16     "23 TSHIANGU, Bonka Elengi, RD CONGO");
17
18 // Creation de la facture
19 $facture = new Facture("FAC-2024-001", $client, 0.16);
20
21 // Ajout des lignes de facture
22 $facture->ajouterLigne(new LigneFacture($produit1, 2));
23 $facture->ajouterLigne(new LigneFacture($produit2, 1));
24 $facture->ajouterLigne(new LigneFacture($produit3, 1));
25
26 // Affichage de la facture
27 $facture->afficher();

```

Listing 12 – Fichier index.php — Utilisation de toutes les classes

Resultat attendu :

```
1 =====
2 FACTURE N : FAC-2024-001
3 Date      : 15/01/2024
4 Client    : Ezechiel ITEWE
5 Adresse   : 23 TSHIANGU, Bonka Elengi, RD CONGO
6 -----
7 Produit   | Qte | Prix U. | Sous-total
8 -----
9 Clavier mecanique | 2 | 89.99 CDF | 179.98 CDF
10 Souris sans fil | 1 | 45.50 CDF | 45.50 CDF
11 Ecran 27 pouces | 1 | 349.00 CDF | 349.00 CDF
12 -----
13 Total HT : 574.48 CDF
14 TVA (16%) : 114.90 CDF
15 -----
16 TOTAL TTC : 689.38 CDF
17 =====
```

Listing 13 – Sortie de la facture

6 Etape 5 : Encapsulation

6.1 Concept : L'encapsulation

Definition. L'encapsulation est le principe qui consiste a restreindre l'accès direct aux attributs d'un objet depuis l'exterieur de la classe. Plutot que d'accéder directement aux attributs, on passe par des methodes speciales appelees **accesseurs** (getters) et **mutateurs** (setters).

Pourquoi encapsuler ?

- Protéger l'intégrité des données : empêcher des valeurs absurdes (un prix négatif, par exemple)
- Contrôler ce qui est lisible et ce qui est modifiable
- Pouvoir changer l'implémentation interne sans impacter le code externe

Les modificateurs de visibilité en PHP :

- **public** : accessible depuis n'importe où
- **private** : accessible uniquement depuis l'intérieur de la classe
- **protected** : accessible depuis la classe et ses classes filles (héritage)

6.2 Application : Produit avec encapsulation

```
1 <?php
2 // Fichier : Produit.php
3
4 class Produit
5 {
6     // Les attributs sont désormais privés.
7     // Ils ne sont plus accessibles directement depuis l'extérieur.
8     private string $nom;
9     private float $prixUnitaire;
10    private int $quantiteStock;
11
12    public function __construct(
13        string $nom,
14        float $prixUnitaire,
15        int $quantiteStock = 0
16    ) {
17        // On utilise les setters pour beneficier de la validation
18        $this->setNom($nom);
19        $this->setPrixUnitaire($prixUnitaire);
20        $this->setQuantiteStock($quantiteStock);
21    }
22
23    // ---- Getters (accesseurs) ----
24
25    public function getNom(): string
26    {
27        return $this->nom;
28    }
29
30    public function getPrixUnitaire(): float
```

```
31 {
32     return $this->prixUnitaire;
33 }
34
35 public function getQuantiteStock(): int
36 {
37     return $this->quantiteStock;
38 }
39
40 // ---- Setters (mutateurs) avec validation ----
41
42 public function setNom(string $nom): void
43 {
44     // Le nom ne doit pas etre vide
45     if (empty(trim($nom))) {
46         throw new InvalidArgumentException(
47             "Le nom du produit ne peut pas etre vide."
48         );
49     }
50     $this->nom = trim($nom);
51 }
52
53 public function setPrixUnitaire(float $prix): void
54 {
55     // Le prix ne peut pas etre negatif
56     if ($prix < 0) {
57         throw new InvalidArgumentException(
58             "Le prix unitaire ne peut pas etre negatif."
59         );
60     }
61     $this->prixUnitaire = $prix;
62 }
63
64 public function setQuantiteStock(int $quantite): void
65 {
66     // La quantite ne peut pas etre negative
67     if ($quantite < 0) {
68         throw new InvalidArgumentException(
69             "La quantite en stock ne peut pas etre negative."
70         );
71     }
72     $this->quantiteStock = $quantite;
73 }
74
75 // ---- Methodes metier ----
76
77 public function afficherInfos(): void
78 {
79     echo "Produit : " . $this->nom . "\n";
80     echo "Prix unitaire : " . $this->prixUnitaire . " CDF\n";
81     echo "Stock : " . $this->quantiteStock . " unites\n";
```



```
82     }
83
84     public function valeurStock(): float
85     {
86         return $this->prixUnitaire * $this->quantiteStock;
87     }
88 }
```

Listing 14 – Fichier Produit.php — Version encapsulee

6.3 Demonstration de la protection offerte par l'encapsulation

```
1 <?php
2 require_once 'Produit.php';
3
4 $produit = new Produit("Clavier mecanique", 89.99, 15);
5
6 // Acces correct via le getter
7 echo $produit->getNom() . "\n"; // Affiche : Clavier mecanique
8
9 // Modification correcte via le setter
10 $produit->setPrixUnitaire(79.99);
11 echo $produit->getPrixUnitaire() . "\n"; // Affiche : 79.99
12
13 // Tentative d'accès direct a un attribut prive
14 // La ligne suivante genere une erreur fatale :
15 // $produit->prixUnitaire = 10;
16 // Fatal error: Cannot access private property Produit::$prixUnitaire
17
18 // Le setter protege contre les valeurs invalides
19 try {
20     $produit->setPrixUnitaire(-50); // Prix negatif : interdit
21 } catch (InvalidArgumentException $e) {
22     echo "Erreur : " . $e->getMessage() . "\n";
23     // Affiche : Erreur : Le prix unitaire ne peut pas etre negatif.
24 }
```

Listing 15 – Demonstration de l'encapsulation dans index.php

7 Etape 6 : Heritage

7.1 Concept : L'heritage

Definition. L'heritage est un mecanisme qui permet a une classe (classe fille ou sous-classe) d'heriter des attributs et des methodes d'une autre classe (classe mere ou superclasse). Cela permet de reutiliser du code et de creer des hierarchies de classes.

En PHP, l'heritage se declare avec le mot-cle `extends`.

Regles importantes :

- Une classe fille herite de tous les membres `public` et `protected` de la classe mere
- Elle peut ajouter de nouveaux attributs et methodes
- Elle peut redefinir (surcharger) les methodes de la classe mere
- Pour appeler le constructeur de la classe mere depuis la classe fille, on utilise `parent::__construct()`

7.2 Application : FactureDetailee herite de Facture

Nous allons creer une classe `FactureDetailee` qui herite de `Facture` et ajoute la gestion d'une remise commerciale.

```
1 <?php
2 // Fichier : FactureDetailee.php
3
4 // On inclut la classe mere
5 require_once 'Facture.php';
6
7 /**
8  * FactureDetailee etend Facture en ajoutant une remise commerciale.
9  * Elle herite de toutes les proprietes et methodes de Facture.
10 */
11 class FactureDetailee extends Facture
12 {
13     // Nouvel attribut specifique a cette classe fille
14     private float $remise; // Remise en pourcentage (ex: 0.10 pour 10%)
15     private string $motifRemise;
16
17     /**
18      * Constructeur de FactureDetailee.
19      * On appelle le constructeur de la classe mere avec parent::__construct
20      */
21     public function __construct(
22         string $numero,
23         Client $client,
24         float $tauxTva = 0.16,
25         float $remise = 0.0,
26         string $motifRemise = ""
27     ) {
```

```
28      // Appel du constructeur de Facture (classe mere)
29      parent::__construct($numero, $client, $tauxTva);
30
31      // Initialisation des attributs propres a FactureDetaillee
32      $this->remise      = $remise;
33      $this->motifRemise = $motifRemise;
34  }
35
36      // ---- Getters pour les nouveaux attributs ----
37
38      public function getRemise(): float
39      {
40          return $this->remise;
41      }
42
43      public function getMotifRemise(): string
44      {
45          return $this->motifRemise;
46      }
47
48      /**
49       * Calcule le montant de la remise sur le total HT
50       */
51      public function calculerMontantRemise(): float
52      {
53          // On utilise la methode heritee calculerTotalHT()
54          return $this->calculerTotalHT() * $this->remise;
55      }
56
57      /**
58       * Calcule le total HT apres remise
59       */
60      public function calculerTotalHTApresRemise(): float
61      {
62          return $this->calculerTotalHT() - $this->calculerMontantRemise();
63      }
64
65      /**
66       * Surcharge (redefinition) de calculerTotalTTC()
67       * pour prendre en compte la remise
68       */
69      public function calculerTotalTTC(): float
70      {
71          $htApresRemise = $this->calculerTotalHTApresRemise();
72          return $htApresRemise + ($htApresRemise * $this->tauxTva);
73      }
74
75      /**
76       * Surcharge de afficher() pour inclure la remise
77       */
78      public function afficher(): void
```

```

79 {
80     echo str_repeat("=", 65) . "\n";
81     echo "FACTURE DETAILLEE N : " . $this->numero . "\n";
82     echo "Date      : " . $this->dateCreation . "\n";
83     echo "Client    : " . $this->client->getNomComple() . "\n";
84     echo str_repeat("-", 65) . "\n";
85     echo sprintf(
86         "%-30s | %5s | %12s | %12s\n",
87         "Produit", "Qte", "Prix U.", "Sous-total"
88     );
89     echo str_repeat("-", 65) . "\n";
90
91     foreach ($this->lignes as $ligne) {
92         $ligne->afficher();
93     }
94
95     echo str_repeat("-", 65) . "\n";
96     echo sprintf("%-48s %10.2f CDF\n",
97         "Total HT :", $this->calculerTotalHT());
98
99     if ($this->remise > 0) {
100         echo sprintf("%-48s %10.2f CDF\n",
101             "Remise (" . ($this->remise * 100) . "% - " . $this->
motifRemise . ") :",
102             -$this->calculerMontantRemise());
103         echo sprintf("%-48s %10.2f CDF\n",
104             "Total HT apres remise :", $this->calculerTotalHTApresRemise
105             ());
106     }
107
108     $htBase = $this->calculerTotalHTApresRemise();
109     echo sprintf("%-48s %10.2f CDF\n",
110         "TVA (" . ($this->tauxTva * 100) . "%) :",
111         $htBase * $this->tauxTva);
112     echo str_repeat("-", 65) . "\n";
113     echo sprintf("%-48s %10.2f CDF\n",
114         "TOTAL TTC :", $this->calculerTotalTTC());
115     echo str_repeat("=", 65) . "\n";
116 }

```

Listing 16 – Fichier FactureDetaillee.php — Heritage de Facture

Remarque sur la visibilite. Dans la classe `Facture`, les attributs `$numero`, `$client`, `$lignes`, `$dateCreation` et `$tauxTva` doivent etre declares `public` ou `protected` pour etre accessibles depuis la classe fille `FactureDetaillee`. Un attribut `private` n'est accessible que dans la classe qui le definit.

7.3 Utilisation

```
1 <?php
2 require_once 'FactureDetailllee.php';
3
4 $client = new Client("Ezechiel", "Itewe", "ezechiel.itewe@email.com",
5                     "23 TSHIANGU, Bonka Elengi, RD CONGO");
6
7 $produit1 = new Produit("Ordinateur portable", 750.00, 5);
8 $produit2 = new Produit("Sacoche", 35.00, 20);
9
10 // Creation d'une facture detaillee avec une remise de 10%
11 $facture = new FactureDetailllee(
12     "FAC-2024-002",
13     $client,
14     0.16,          // TVA a 16%
15     0.10,          // Remise de 10%
16     "Client fidele"
17 );
18
19 $facture->ajouterLigne(new LigneFacture($produit1, 1));
20 $facture->ajouterLigne(new LigneFacture($produit2, 2));
21
22 // La methode afficher() est la version surchargee de FactureDetailllee
23 $facture->afficher();
```

Listing 17 – Utilisation de FactureDetailllee

8 Etape 7 : Polymorphisme

8.1 Concept : Le polymorphisme

Definition. Le polymorphisme est la capacite pour des objets de classes differentes de repondre de maniere differente a un meme appel de methode. En d'autres termes, une meme methode peut avoir des comportements differents selon l'objet sur lequel elle est appelee.

Il existe deux formes principales de polymorphisme en PHP :

- **La surcharge (overriding)** : une classe fille redefinit une methode de la classe mere. C'est ce que nous avons fait avec `afficher()` et `calculerTotalTTC()` dans `FactureDetaillee`.
- **Le polymorphisme par interface ou classe abstraite** : plusieurs classes non liees par heritage direct implementent les memes methodes.

Le polymorphisme permet d'ecrire du code generique qui fonctionne avec n'importe quel objet d'un type donne, sans avoir a se preoccuper de son type exact.

8.2 Application : Demonstration du polymorphisme

Nous allons creer plusieurs types de factures et les traiter de maniere uniforme :

```

1 <?php
2 // Fichier : index.php
3
4 require_once 'Facture.php';
5 require_once 'FactureDetaillee.php';
6
7 // Fonction generique qui accepte n'importe quelle Facture
8 // (ou classe qui en herite)
9 function traiterFacture(Facture $facture): void
10 {
11     // Appel identique, mais comportement different
12     // selon le type reel de l'objet
13     $facture->afficher();
14     echo "Total TTC calcule : " . $facture->calculerTotalTTC() . " CDF\n\n";
15 }
16
17 // ---- Creation des donnees ----
18 $produit1 = new Produit("Serveur", 1200.00, 2);
19 $produit2 = new Produit("Switch reseau", 350.00, 3);
20
21 $client1 = new Client("Ezechiel", "Itewe", "ezechiel.itewe@email.com",
22     "78 rue du Commerce, RDC Kinshasa");
23 $client2 = new Client("Princesse", "Kalala", "princesse.kalala@email.com",
24     "23 boulevard du 30 juin, Congo Kinshasa");
25
26 // Facture standard (classe mere)
27 $factureStandard = new Facture("FAC-2024-010", $client1, 0.18);
28 $factureStandard->ajouterLigne(new LigneFacture($produit1, 1));
29 $factureStandard->ajouterLigne(new LigneFacture($produit2, 2));

```

```
30
31 // Facture detaillee avec remise (classe fille)
32 $factureDetaillee = new FactureDetaillee(
33     "FAC-2024-011",
34     $client2,
35     0.16,
36     0.15,
37     "Commande superieure a 1000 CDF"
38 );
39 $factureDetaillee->ajouterLigne(new LigneFacture($produit1, 2));
40 $factureDetaillee->ajouterLigne(new LigneFacture($produit2, 1));
41
42 // Tableau contenant des objets de types differents
43 // mais partageant la meme classe de base : Facture
44 $factures = [$factureStandard, $factureDetaillee];
45
46 // Traitement uniforme : le polymorphisme fait que
47 // chaque objet utilise sa propre version de afficher()
48 // et de calculerTotalTTC()
49 foreach ($factures as $facture) {
50     traiterFacture($facture);
51     echo str_repeat("*", 65) . "\n\n";
52 }
```

Listing 18 – Demonstration du polymorphisme sur les factures

Explication. La fonction `traiterFacture()` recoit un parametre de type `Facture`. Elle peut donc recevoir un objet `Facture` ou tout objet d'une classe qui herite de `Facture` (comme `FactureDetaillee`). Lorsqu'elle appelle `$facture->afficher()`, PHP determine automatiquement quelle version de la methode appeler selon le type reel de l'objet. C'est le polymorphisme.

9 Etape 8 : Abstraction

9.1 Concept : Les classes abstraites et les interfaces

Classe abstraite. Une classe abstraite est une classe qui ne peut pas être instanciée directement. Elle sert uniquement de modèle pour d'autres classes. Elle peut contenir des méthodes abstraites (sans implémentation, juste la signature) que les classes filles doivent obligatoirement implémenter. Elle peut aussi contenir des méthodes concrètes (avec implémentation).

Interface. Une interface est un contrat que les classes s'engagent à respecter. Elle ne contient que des signatures de méthodes, sans implémentation. Une classe peut implémenter plusieurs interfaces (contrairement à l'héritage simple).

Mot-cle `abstract` : Une classe est déclarée abstraite avec `abstract class`. Une méthode abstraite est déclarée avec `abstract function`.

9.2 Application : Entité abstraite commune

Nous allons créer une classe abstraite `EntiteAbstraite` qui impose à toutes les entités du système (Produit, Client, Facture) d'avoir une méthode `afficherInfos()` et une méthode pour obtenir un identifiant unique.

```
1 <?php
2 // Fichier : EntiteAbstraite.php
3
4 /**
5  * Classe abstraite de base pour toutes les entites du systeme.
6  * Elle ne peut pas etre instanciee directement.
7  */
8 abstract class EntiteAbstraite
9 {
10     // Attribut commun a toutes les entites
11     protected string $identifiant;
12     protected string $dateCreation;
13
14     /**
15      * Le constructeur est protected : il ne peut etre appele
16      * que depuis les constructeurs des classes filles.
17      */
18     protected function __construct(string $identifiant)
19     {
20         $this->identifiant = $identifiant;
21         $this->dateCreation = date('d/m/Y H:i:s');
22     }
23
24     /**
25      * Methode concrete : implementee ici, disponible dans toutes
26      * les classes filles sans qu'elles aient a la redefinir.
27      */
28     public function getIdentifiant(): string
29     {
```



```

30     return $this->identifiant;
31 }
32
33 public function getDateCreation(): string
34 {
35     return $this->dateCreation;
36 }
37
38 /**
39  * Methode abstraite : TOUTES les classes filles DOIVENT
40  * implementer cette methode. Si elles ne le font pas,
41  * PHP genere une erreur fatale.
42  */
43 abstract public function afficherInfos(): void;
44
45 /**
46  * Methode abstraite : chaque entite doit savoir
47  * se représenter sous forme de chaine de caracteres.
48  */
49 abstract public function toString(): string;
50 }

```

Listing 19 – Fichier EntiteAbstraite.php — Classe abstraite

9.3 Application : Produit et Client heritent de EntiteAbstraite

```

1 <?php
2 // Fichier : Produit.php
3
4 require_once 'EntiteAbstraite.php';
5
6 class Produit extends EntiteAbstraite
7 {
8     private string $nom;
9     private float $prixUnitaire;
10    private int $quantiteStock;
11
12    public function __construct(
13        string $nom,
14        float $prixUnitaire,
15        int $quantiteStock = 0
16    ) {
17        // Appel du constructeur de la classe abstraite
18        // L'identifiant est genere automatiquement
19        parent::__construct('PRD-' . uniqid());
20
21        $this->setNom($nom);
22        $this->setPrixUnitaire($prixUnitaire);
23        $this->setQuantiteStock($quantiteStock);
24    }
25
26    // ---- Getters ----

```

```

27     public function getNom(): string        { return $this->nom; }
28     public function getPrixUnitaire(): float { return $this->prixUnitaire; }
29     public function getQuantiteStock(): int  { return $this->quantiteStock; }
30
31     // ---- Setters avec validation ----
32     public function setNom(string $nom): void
33     {
34         if (empty(trim($nom))) {
35             throw new InvalidArgumentException("Le nom ne peut pas etre vide
36             .");
37         }
38         $this->nom = trim($nom);
39     }
40
41     public function setPrixUnitaire(float $prix): void
42     {
43         if ($prix < 0) {
44             throw new InvalidArgumentException("Le prix ne peut pas etre
45             negatif.");
46         }
47         $this->prixUnitaire = $prix;
48     }
49
50     public function setQuantiteStock(int $quantite): void
51     {
52         if ($quantite < 0) {
53             throw new InvalidArgumentException("La quantite ne peut pas etre
54             negative.");
55         }
56         $this->quantiteStock = $quantite;
57     }
58
59     // ---- Implementation des methodes abstraites ----
60
61     /**
62     * Implementation obligatoire de afficherInfos() (methode abstraite)
63     */
64     public function afficherInfos(): void
65     {
66         echo "Produit      : " . $this->nom . "\n";
67         echo "ID           : " . $this->identifiant . "\n";
68         echo "Prix U.      : " . number_format($this->prixUnitaire, 2) . "
69         EUR\n";
70         echo "Stock        : " . $this->quantiteStock . " unites\n";
71         echo "Cree le       : " . $this->dateCreation . "\n";
72     }
73
74     /**
75     * Implementation obligatoire de toString() (methode abstraite)
76     */

```

```

73     public function toString(): string
74     {
75         return sprintf(
76             "[Produit] %s (ID: %s) - %.2f CDF",
77             $this->nom,
78             $this->identifiant,
79             $this->prixUnitaire
80         );
81     }
82
83     public function valeurStock(): float
84     {
85         return $this->prixUnitaire * $this->quantiteStock;
86     }
87 }

```

Listing 20 – Fichier Produit.php — Heritage de EntiteAbstraite

```

1  <?php
2  // Fichier : Client.php
3
4  require_once 'EntiteAbstraite.php';
5
6  class Client extends EntiteAbstraite
7  {
8      private string $nom;
9      private string $prenom;
10     private string $email;
11     private string $adresse;
12
13     public function __construct(
14         string $nom,
15         string $prenom,
16         string $email,
17         string $adresse
18     ) {
19         parent::__construct('CLI-' . uniqid());
20         $this->nom = $nom;
21         $this->prenom = $prenom;
22         $this->email = $email;
23         $this->adresse = $adresse;
24     }
25
26     // ---- Getters ----
27     public function getNom(): string { return $this->nom; }
28     public function getPrenom(): string { return $this->prenom; }
29     public function getEmail(): string { return $this->email; }
30     public function getAdresse(): string { return $this->adresse; }
31
32     public function getNomComplet(): string
33     {
34         return $this->prenom . " " . $this->nom;

```

```

35     }
36
37     // ---- Implementation des methodes abstraites ----
38
39     public function afficherInfos(): void
40     {
41         echo "Client   : " . $this->getNomComplet() . "\n";
42         echo "ID       : " . $this->identifiant . "\n";
43         echo "Email    : " . $this->email . "\n";
44         echo "Adresse  : " . $this->adresse . "\n";
45         echo "Cree le  : " . $this->dateCreation . "\n";
46     }
47
48     public function toString(): string
49     {
50         return sprintf(
51             "[Client] %s (ID: %s) - %s",
52             $this->getNomComplet(),
53             $this->identifiant,
54             $this->email
55         );
56     }
57 }

```

Listing 21 – Fichier Client.php — Heritage de EntiteAbstraite

9.4 Demonstration de l'abstraction

```

1  <?php
2  require_once 'Produit.php';
3  require_once 'Client.php';
4
5  // Tentative d'instanciation directe de la classe abstraite
6  // La ligne suivante genere une erreur fatale :
7  // $entite = new EntiteAbstraite('test');
8  // Fatal error: Cannot instantiate abstract class EntiteAbstraite
9
10 // On peut en revanche instancier les classes concretes
11 $produit = new Produit("Disque SSD 1To", 129.99, 10);
12 $client  = new Client("Ezechiele", "Itewe", "ezechiele.itewe@email.com",
13                      "23 TSHIANGU, Bonka Elengi, RD CONGO");
14
15 // Traitement generique : on sait que les deux objets
16 // possedent la methode afficherInfos() et toString()
17 $entites = [$produit, $client];
18
19 foreach ($entites as $entite) {
20     echo $entite->toString() . "\n";
21     $entite->afficherInfos();
22     echo "ID : " . $entite->getIdentifiant() . "\n";
23     echo str_repeat("-", 50) . "\n";
24 }

```

Listing 22 – Demonstration de la classe abstraite

10 Recapitulatif des concepts

Le tableau suivant recapitule les concepts de la POO vus dans ce cas pratique :

Concept	Definition	Application dans le projet
Classe	Modele definissant des attributs et des methodes	Produit, Client, Facture
Objet	Instance d'une classe	<code>\$produit1 = new Produit(...)</code>
Attribut	Variable propre a un objet	<code>\$nom</code> , <code>\$prixUnitaire</code>
Methode	Fonction propre a une classe	<code>afficherInfos()</code> , <code>calculerTotalHT()</code>
Constructeur	Methode d'initialisation appelee a la creation	<code>__construct()</code> dans chaque classe
Encapsulation	Controle de l'acces aux attributs	Attributs <code>private</code> + <code>getters/setters</code>
Heritage	Reutilisation et extension d'une classe	<code>FactureDetaillee extends Facture</code>
Polymorphisme	Un meme appel produit des comportements differents	<code>afficher()</code> et <code>calculerTotalTTC()</code> surcharges
Abstraction	Definir un contrat oblige les classes filles a l'implementer	<code>abstract class EntiteAbstraite</code>

11 Structure finale du projet

La structure finale du projet, avec les dependances entre fichiers, est la suivante :

```

1 facturation/
2 |-- EntiteAbstraite.php    (classe abstraite de base)
3 |-- Produit.php           (extends EntiteAbstraite)
4 |-- Client.php            (extends EntiteAbstraite)
5 |-- LigneFacture.php       (depend de Produit.php)
6 |-- Facture.php            (depend de Client.php et LigneFacture.php)
7 |-- FactureDetaillee.php   (extends Facture)
8 |-- index.php              (fichier d'execution principal)

```

Listing 23 – Structure finale du projet

Chaine de dependances :

```

EntiteAbstraite ← Produit ← LigneFacture ← Facture ← FactureDetaillee
EntiteAbstraite ← Client ← Facture

```